

Employee Management System

Technical Documentation & Code Walkthrough

Java Swing Application | File-based Persistence | Multi-Role System

7 Classes

4 Roles

6 Data Files

Java Swing UI

Table of Contents

01	System Overview	Architecture, Components, Data Flow
02	Main.java	Entry Point – Application Launch
03	LoginScreen.java	Authentication & Role-Based Routing
04	ManageFile.java	Data Persistence Layer (File I/O)
05	EmployeeDashboard.java	Employee Features & Actions
06	LeaderDashboard.java	Team Leader Features
07	PMDashboard.java	Project Manager Features
08	AdminDashboard.java	Admin – User Management
09	Data Files	6 .dat files explained
10	Summary & Flow	Full System Flow Diagram

System Overview – Architecture & Components

UI Layer (Java Swing – JFrame)

LoginScreen

EmployeeDashboard

LeaderDashboard

PMDashboard

AdminDashboard



Business Logic Layer

ManageFile.java – All Operations



Data Layer (Binary .dat Files)

Tasks.dat

CompletedTasks.dat

WorkLogs.dat

Vacations.dat

Penalties.dat

Data.dat

Main.java – Application Entry Point

```
package org.example;

public class Main {
    public static void main(String[] args) {
        new LoginScreen(); // Launch the app
    }
}
```

What it does

Main.java is the single entry point of the entire application.

It has only one job: create an instance of LoginScreen which opens the login window.

Key Concepts:

Entry Point

Java programs always start from public static void main(String[] args)

Minimal Responsibility

Main only starts the UI – all logic lives elsewhere (separation of concerns)

No Swing.invokeLater

LoginScreen is created directly – works but best practice is to use SwingUtilities.invokeLater()

LoginScreen.java – UI Setup & Components

```
public class LoginScreen extends JFrame {  
    public LoginScreen() {  
        setSize(400, 350);  
        setTitle("Registration");  
  
        JPanel panel = new JPanel();  
        panel.setLayout(new GridLayout(5, 2, 10, 10));  
  
        JLabel labelName = new JLabel("Enter your name");  
        JTextField nameField = new JTextField(20);  
  
        JLabel labelEmail = new JLabel("Enter your Email");  
        JTextField emailField = new JTextField(20);  
  
        JLabel labelRole = new JLabel("Enter your Role");  
        JTextField roleField = new JTextField(20);  
  
        JLabel labelPass = new JLabel("Enter your password");  
        JPasswordField pass = new JPasswordField(20);  
  
        JButton loginBtn = new JButton("Login");  
        ...  
    }  
}
```

UI Components Used:

JFrame	Main window container
JPanel	Holds all UI elements
GridLayout	5 rows × 2 cols grid
JLabel	Text labels for fields
JTextField	Text input boxes
JPasswordField	Masked password input
JButton	Login trigger button

LoginScreen.java – Login Logic & Role Routing

```
loginBtn.addActionListener(_ -> {  
    String role      = roleField.getText().toLowerCase();  
    String password = new String(pass.getPassword());  
  
    if (role.contains("emp") && password.equals("emp123")) {  
        new EmployeeDashboard();  
    } else if (role.contains("leader") && password.equals("leader123")) {  
        new LeaderDashboard();  
    } else if (role.contains("pm") && password.equals("pm123")) {  
        new PMDashboard();  
    } else if (role.contains("admin") && password.equals("admin123")) {  
        new AdminDashboard();  
    } else {  
        JOptionPane.showMessageDialog(this, "Invalid Login");  
        return;  
    }  
    dispose(); // Close login window  
});
```

Role → Password → Screen Mapping:

Role: "emp"

Pass: emp123 → EmployeeDashboard

Role: "leader"

Pass: leader123 → LeaderDashboard

Role: "pm"

Pass: pm123 → PMDashboard

Role: "admin"

Pass: admin123 → AdminDashboard

ManageFile.java – The Data Layer (Overview)

ManageFile is the backbone of the system. Every dashboard uses it to read and write data.

```
private final String TASKS_FILE      = "Tasks.dat";  
private final String COMPLETED_FILE = "CompletedTasks.dat";  
private final String LOGS_FILE       = "WorkLogs.dat";  
private final String VACATION_FILE   = "Vacations.dat";  
private final String PENALTY_FILE    = "Penalties.dat";  
private final String REPORTS_FILE    = "Reports.dat";  
private final String USER_FILE       = "Data.dat";
```

File Format: Binary (DataOutputStream / DataInputStream)
Each file stores typed values: writeUTF(), writeDouble(), writeLong()
Append mode (true) adds records – false mode overwrites for updates.

Task Management

saveTask(), getAllTasks(),
moveTaskToCompleted()

Attendance / Hours

saveWorkLog(),
getMonthlyHours()

Vacation Handling

requestVacation(),
getVacationRequests(),
processVacation()

Reports & Penalties

getEmployeePenalties(),
saveEmployeeReport(),
getAllEmployeeReports()

Project Status

getProjectStatusReport(),
getPMPProjectReport(),
getCompletionPercentage()

User Management

addUser(),
updateOrDeleteUser()

ManageFile.java – Task Management Functions

```
// Save a new task to Tasks.dat
public void saveTask(String task) {
    try (DataOutputStream out = new DataOutputStream(
        new FileOutputStream(TASKS_FILE, true))) {
        out.writeUTF(task);    // Write as UTF string
    } catch (IOException e) { e.printStackTrace(); }
}

// Read all pending tasks from file
public List<String> getAllTasks() {
    List<String> taskList = new ArrayList<>();
    File file = new File(TASKS_FILE);
    if (!file.exists()) return taskList;

    try (DataInputStream in = new DataInputStream(
        new FileInputStream(file))) {
        while (in.available() > 0)
            taskList.add(in.readUTF());
    } catch (IOException e) { e.printStackTrace(); }
    return taskList;
}
```

How it works:

FileOutputStream(file, true)

The 'true' flag = APPEND mode. New tasks are added at end of file without erasing old ones.

DataOutputStream.writeUTF()

Writes string with a 2-byte length prefix, making it easy to read back safely.

in.available() > 0

Loop until no more bytes left in file – reads every stored string.

try-with-resources

The (DataOutputStream out = ...) syntax auto-closes the file even if error occurs.

ManageFile.java – Move Task & Vacation Handling

```
public void moveTaskToCompleted(String task) {
    List<String> remaining = getAllTasks();
    if (remaining.remove(task)) {
        // Rewrite Tasks.dat without the completed task
        try (DataOutputStream out = new DataOutputStream(
            new FileOutputStream(TASKS_FILE, false))) {
            for (String t : remaining) out.writeUTF(t);
        } catch (IOException e) { e.printStackTrace(); }

        // Append to CompletedTasks.dat
        try (DataOutputStream out = new DataOutputStream(
            new FileOutputStream(COMPLETED_FILE, true))) {
            out.writeUTF(task);
        } catch (IOException e) { e.printStackTrace(); }
    }
}
```

```
public void requestVacation(String emp, String reason) {
    try (DataOutputStream out = new DataOutputStream(
        new FileOutputStream(VACATION_FILE, true))) {
        out.writeUTF(emp);
        out.writeUTF(reason);
        out.writeUTF("Pending"); // Default status
    } catch (IOException e) { e.printStackTrace(); }
}
```

Delete-then-Rewrite Pattern

Since binary files can't delete a middle record, the trick is:

1. Read ALL records into a list
2. Remove the unwanted one
3. Rewrite the WHOLE file from scratch (false = overwrite)

This is used for tasks, vacations, and users.

Vacation Record Structure

Each vacation = 3 values written in sequence:

- Employee name (UTF)
- Reason (UTF)
- Status: 'Pending' / 'Accepted' / 'Rejected'

The processVacation() method reads all, updates status, and rewrites.

EmployeeDashboard.java – UI Layout & Buttons

```
public class EmployeeDashboard extends JFrame {
    private final ManageFile fileHandler = new ManageFile();
    private final AtomicLong startTime = new AtomicLong(0);

    public EmployeeDashboard() {
        setTitle("Employee Panel");
        setSize(450, 400);
        setLayout(null);    // Absolute positioning

        JButton checkIn  = new JButton("Check In");
        checkIn.setBounds(50, 30, 150, 30);

        JButton checkOut = new JButton("Check Out");
        checkOut.setBounds(220, 30, 150, 30);

        JButton tasks      = new JButton("View Tasks");
        JButton completeTask = new JButton("Complete Task");
        JButton vacationBtn = new JButton("Request Vacation");
        JButton penaltyBtn  = new JButton("View Penalties");
        JButton viewHoursBtn = new JButton("Monthly Hours");
        JButton viewVacations = new JButton("My Vacations");
        JButton logout      = new JButton("Logout");
        JLabel status       = new JLabel("Status: Idle");
        ...
    }
}
```

Employee Features:

Check In / Out

Tracks working hours using system clock timestamp

View Tasks

Shows all pending tasks assigned by leader

Complete Task

Moves selected task to completed list

Request Vacation

Submits vacation request to Vacations.dat

My Vacations

Shows employee's own vacation history + status

View Penalties

Lists deductions recorded by admin/leader

Monthly Hours

Sums hours logged this calendar month

EmployeeDashboard.java – Check-In / Check-Out Logic

```
// AtomicLong stores the check-in timestamp safely
private final AtomicLong startTime = new AtomicLong(0);

// CHECK IN: record current time
checkIn.addActionListener(_ -> {
    startTime.set(System.currentTimeMillis());
    status.setText("Status: Checked In");
    JOptionPane.showMessageDialog(this, "Clock started!");
});

// CHECK OUT: calculate hours elapsed, save to file
checkOut.addActionListener(_ -> {
    if (startTime.get() == 0) return; // Guard: not checked in

    double hours = (double)(System.currentTimeMillis()
        - startTime.get()) / 3600000; // ms → hours

    fileHandler.saveWorkLog("EmployeeName", hours);
    status.setText(String.format("Logged %.2f hours.", hours));
    startTime.set(0); // Reset timer
});
```

AtomicLong

Used instead of a plain long because it's thread-safe. Swing event handling can run on multiple threads, so AtomicLong prevents race conditions when reading/writing startTime.

Time Calculation

System.currentTimeMillis() returns time in milliseconds.

Formula:

$\text{hours} = (\text{checkout_ms} - \text{checkin_ms}) / 3_600_000$

Because $1 \text{ hour} = 60 \times 60 \times 1000 = 3,600,000 \text{ ms}$

saveWorkLog()

Writes to WorkLogs.dat:

- Employee name (UTF)
- Hours worked (double)
- Timestamp (long)

EmployeeDashboard.java – Tasks & Vacation Code

```
// View all pending tasks
tasks.addActionListener(_ -> {
    List<String> list = fileHandler.getAllTasks();
    JOptionPane.showMessageDialog(this,
        list.isEmpty() ? "No tasks." : String.join("\n", list));
});

// Mark a task as complete
completeTask.addActionListener(_ -> {
    List<String> list = fileHandler.getAllTasks();
    if (list.isEmpty()) return;
    String choice = (String) JOptionPane.showInputDialog(
        this, "Select Task:", "Complete",
        JOptionPane.QUESTION_MESSAGE, null,
        list.toArray(), list.getFirst());
    if (choice != null) fileHandler.moveTaskToCompleted(choice);
});

// Request a vacation
vacationBtn.addActionListener(_ -> {
    String reason = JOptionPane.showInputDialog(this, "Reason:");
    if (reason != null)
        fileHandler.requestVacation("EmployeeName", reason);
});
```

showInputDialog with Dropdown

When you pass an array to showInputDialog, Swing automatically shows a dropdown (JComboBox) instead of a text field.

This lets the employee pick from existing tasks.

String.join()

Converts a List<String> into a single string with newlines between items – clean way to display a list in a dialog.

Hardcoded Name Issue

"EmployeeName" is hardcoded in the current code. In a real system, the logged-in user's name would be passed to the dashboard and stored as a field.

LeaderDashboard.java – Team Leader Features

```
// Assign a new task
assignTask.addActionListener(_ -> {
    String task = JOptionPane.showInputDialog(this, "Task description:");
    if (task != null && !task.trim().isEmpty()) {
        fileHandler.saveTask(task);
        JOptionPane.showMessageDialog(this, "Task Assigned!");
    }
});

// Manage vacation requests (Accept / Reject)
manageVacations.addActionListener(_ -> {
    List<String> requests = fileHandler.getVacationRequests();
    if (requests.isEmpty()) {
        JOptionPane.showMessageDialog(this, "No requests found.");
        return;
    }
    String choice = (String) JOptionPane.showInputDialog(
        this, "Select Request:", "Vacation Manager",
        JOptionPane.QUESTION_MESSAGE, null,
        requests.toArray(), requests.get(0));

    if (choice != null) {
        Object[] options = {"Accept", "Reject", "Cancel"};
        int n = JOptionPane.showOptionDialog(this, "Decision:", ...);
        if (n == JOptionPane.YES_OPTION)
            fileHandler.processVacation(choice, "Accepted");
        else if (n == JOptionPane.NO_OPTION)
            fileHandler.processVacation(choice, "Rejected");
    }
});
```

Leader Buttons:

Assign Task

Input dialog → saves to Tasks.dat

Project Overview

Shows pending + completed tasks in scrollable text area

Manage Vacations

Shows dropdown of all vacation requests. Leader can Accept or Reject. Status is updated in Vacations.dat

View PM Reports

Reads Reports.dat written by PM and displays all entries with dates

Logout

Closes window → opens LoginScreen again

LeaderDashboard.java – showOptionDialog Deep Dive

```
Object[] options = {"Accept", "Reject", "Cancel"};
int n = JOptionPane.showOptionDialog(
    this,                      // Parent component
    "What to do with: " + choice, // Message shown to user
    "Decision",                 // Dialog title
    JOptionPane.YES_NO_CANCEL_OPTION, // Option type
    JOptionPane.QUESTION_MESSAGE, // Message icon type
    null,                      // No custom icon
    options,                   // Button labels array
    options[2]                 // Default = "Cancel"
);

// n == 0 → "Accept" clicked (YES_OPTION)
// n == 1 → "Reject" clicked (NO_OPTION)
// n == 2 → "Cancel" clicked (CANCEL_OPTION)

if (n == JOptionPane.YES_OPTION)
    fileHandler.processVacation(choice, "Accepted");
else if (n == JOptionPane.NO_OPTION)
    fileHandler.processVacation(choice, "Rejected");
```

showOptionDialog Return Value

Returns an integer:

- 0 = first button (YES)
- 1 = second button (NO)
- 2 = third button (CANCEL)
- -1 = dialog closed with X

YES_NO_CANCEL_OPTION

Tells Swing what button CONSTANTS to use internally, even though you're supplying custom labels.

YES_OPTION = 0, NO_OPTION = 1, CANCEL_OPTION = 2

processVacation()

Takes the full display string of the chosen request, reads all vacation records, finds matching entry, changes its status to 'Accepted' or 'Rejected', rewrites the file.

PMDashboard.java – Project Manager Panel

```
public class PMDashboard extends JFrame {
    private final ManageFile fileHandler = new ManageFile();

    public PMDashboard() {
        JButton viewProjects = new JButton("View All Projects");
        JButton reportBtn = new JButton("Report to Team Leader");
        JButton logout = new JButton("Logout");

        // View pending and completed projects
        viewProjects.addActionListener(_ -> {
            String report = fileHandler.getPMPProjectReport();
            JTextArea textArea = new JTextArea(report);
            textArea.setFont(new Font("Monospaced", Font.PLAIN, 12));
            JScrollPane scrollPane = new JScrollPane(textArea);
            scrollPane.setPreferredSize(new Dimension(350, 300));
            JOptionPane.showMessageDialog(this, scrollPane,
                "Project Modes", JOptionPane.PLAIN_MESSAGE);
        });

        // Send a report about an employee to Team Leader
        reportBtn.addActionListener(_ -> {
            String empName = JOptionPane.showInputDialog(this, "Target
Employee:");
            if (empName != null && !empName.trim().isEmpty()) {
                String details = JOptionPane.showInputDialog(this, "Details:");
                if (details != null && !details.trim().isEmpty()) {
                    fileHandler.saveEmployeeReport(empName, details);
                }
            }
        });
    }
}
```

PM Responsibilities

- View ALL tasks (pending + completed)
- Write performance reports about employees
- Reports are stored in Reports.dat
- Team Leader can read these reports

getPMPProjectReport()

Reads both Tasks.dat and CompletedTasks.dat and formats them as:

[MODE: PENDING]

- Task 1

[MODE: FINISHED]

/ Task 2

saveEmployeeReport()

Writes 3 values to Reports.dat:

- Employee name (UTF)

AdminDashboard.java – User Management Panel

```
// ADD USER
addUser.addActionListener(_ -> {
    String name = JOptionPane.showInputDialog(this, "Enter Username:");
    String pass = JOptionPane.showInputDialog(this, "Enter Password:");
    String[] roles = {"Employee", "Team Leader", "PM", "Admin"};
    String role = (String) JOptionPane.showInputDialog(this,
        "Select Role:", "Role Selection",
        JOptionPane.QUESTION_MESSAGE, null, roles, roles[0]);
    if (role != null)
        fileHandler.addUser(name, pass, role);
});

// UPDATE OR DELETE USER
manageUsers.addActionListener(_ -> {
    String target = JOptionPane.showInputDialog(this, "Username to Manage:");
    Object[] options = {"Update", "Delete", "Cancel"};
    int choice = JOptionPane.showOptionDialog(...);

    if (choice == YES_OPTION) {          // UPDATE
        String newPass = JOptionPane.showInputDialog(this, "New Password:");
        String newRole = (String) JOptionPane.showInputDialog(...);
        fileHandler.updateOrDeleteUser(target, newPass, newRole, false);

    } else if (choice == NO_OPTION) {    // DELETE
        int confirm = JOptionPane.showConfirmDialog(this, "Are you sure?");
        if (confirm == YES_OPTION)
            fileHandler.updateOrDeleteUser(target, "", "", true);
    }
});
```

Admin Capabilities

- View project status (same as PM)
- Add new users to Data.dat
- Update any user's password/role
- Delete users from the system

updateOrDeleteUser()

Parameters:

- targetUser: username to find
- newPass / newRole: new values
- isDelete: true = remove, false = update

Reads all users, modifies/skips target, rewrites file.

Role Dropdown

showInputDialog with a String[] array shows a dropdown (JComboBox) automatically. No custom code needed.

Data Files – All 6 .dat Files Explained

Tasks.dat

Format: writeUTF(task)

Written by Leader, Read by Employee & Leader

CompletedTasks.dat

Format: writeUTF(task)

Written by Employee (on complete), Read by Leader & PM

WorkLogs.dat

Format: writeUTF(name)
 writeDouble(hours)
 writeLong(timestamp)

Written by Employee (checkout), Read by Employee (hours)

Vacations.dat

Format: writeUTF(emp)
 writeUTF(reason)
 writeUTF(status)

Written by Employee, Updated by Leader, Read by Employee & Leader

Penalties.dat

Format: writeUTF(name)
 writeUTF(reason)
 writeDouble(amount)

Written externally/manually, Read by Employee

Data.dat

Format: writeUTF(username)
 writeUTF(password)
 writeUTF(role)

Written & managed by Admin

Key Java & Swing Concepts Used in This Project

JFrame

Top-level window. Each dashboard extends JFrame to become a standalone window.

setLayout(null)

Disables layout manager – all positions set manually using setBounds(x,y,w,h).

DataOutputStream/InputStream

Binary I/O streams for typed data: writeUTF, writeDouble, writeLong, readUTF...

SwingUtilities.invokeLater

Ensures UI updates happen on the Event Dispatch Thread (EDT) for thread safety.

dispose()

Closes and releases the current JFrame window from memory.

JOptionPane

Built-in dialog toolkit: showMessageDialog, showInputDialog, showOptionDialog.

ActionListener (lambda)

btn.addActionListener(_ -> {...}) – modern lambda syntax for event handling.

try-with-resources

try (Stream s = ...) automatically closes the stream when done – prevents leaks.

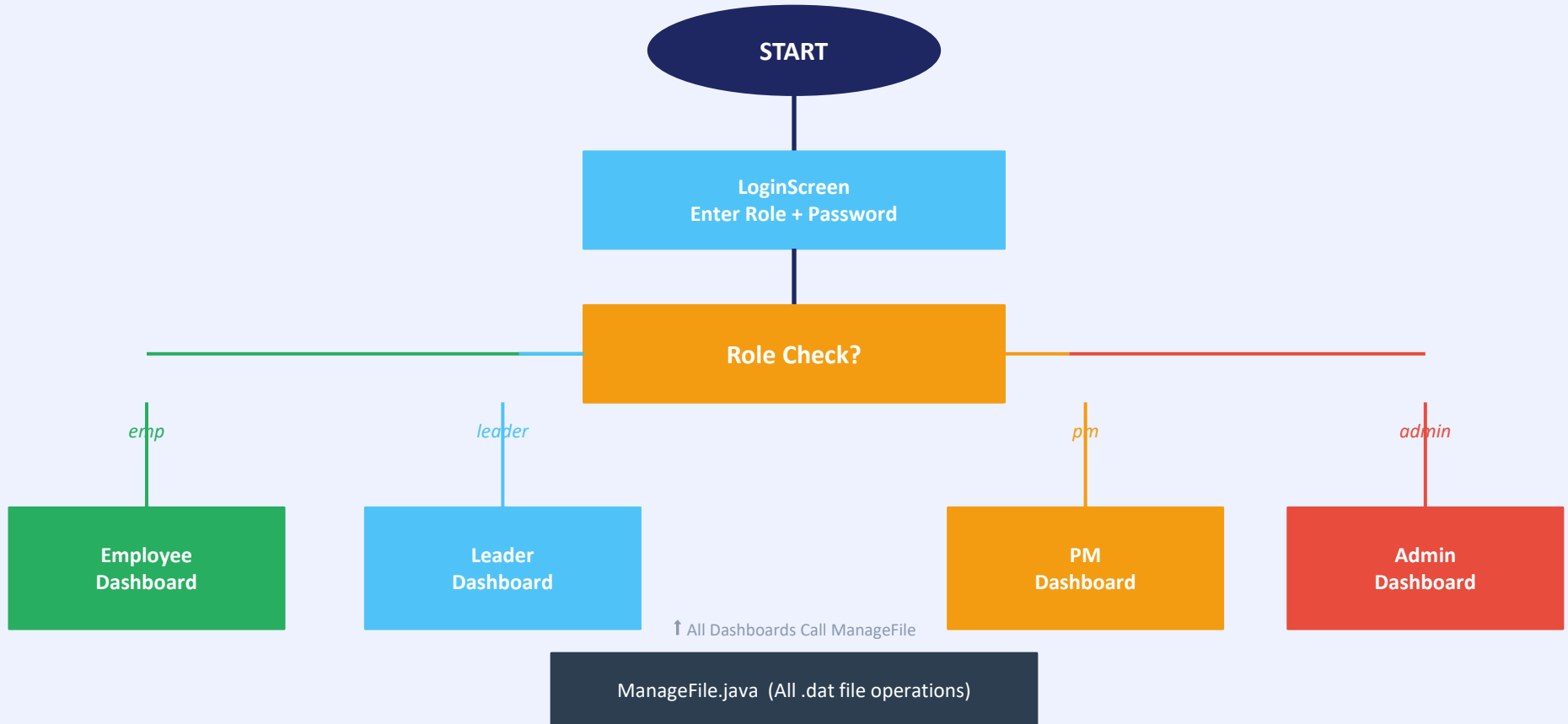
AtomicLong

Thread-safe long variable. Used for startTime in EmployeeDashboard.

JScrollPane

Wraps any component (like JTextArea) to add scroll bars when content is large.

Full System Flow – From Login to Action



Improvements & Best Practices for Future Development

Use a Real Database

Fix: Use JDBC + SQLite

Replace .dat binary files with SQLite or MySQL. Binary files don't support search, indexing, or concurrent access.

Fix Hardcoded 'EmployeeName'

Fix: Pass username in constructor

The current code hardcodes the employee name. Pass the logged-in username as a constructor parameter to each dashboard.

Hash Passwords

Fix: BCrypt / MessageDigest

Passwords stored in plain text in Data.dat. Use BCrypt or SHA-256 to hash passwords before storing.

Use Layout Managers

Fix: GridBagLayout / MigLayout

setLayout(null) with setBounds() breaks on different screen sizes. Use BorderLayout / GridBagLayout instead.

Add Input Validation

Fix: Validate before writing

Many input dialogs don't validate. Empty strings or wrong formats could cause errors or corrupt files.

SwingUtilities.invokeLater()

Fix: Wrap in invokeLater()

Wrap new LoginScreen() in SwingUtilities.invokeLater() to ensure all UI runs on the Event Dispatch Thread (EDT).

System Summary

7 Java Classes

Main, LoginScreen, ManageFile, EmployeeDashboard, LeaderDashboard, PMDashboard, AdminDashboard

4 User Roles

Employee → Leader → PM → Admin (each with dedicated dashboard)

6 Data Files

Binary .dat files for tasks, logs, vacations, penalties, reports, users

Core Pattern

UI Layer (Swing JFrames) → Business Logic (ManageFile) → Data (.dat files)

I/O Method

DataOutputStream/InputStream with writeUTF/readUTF/writeDouble/writeLong

UI Toolkit

Java Swing: JFrame, JPanel, JButton, JOptionPane, JScrollPane, JTextArea